

Reviewer Pack — Noncommutative Fourier Gap in Read-Twice BPs for IP_2

Entry 713f57619482 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 07:23:47 UTC

Noncommutative Fourier Gap in Read-Twice BPs for IP_2

Entry ID: 713f57619482 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 07:23:47 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Harmonic Analysis **Field B** (complexity object): Read-Twice Branching Programs

Statement:

Let P be a read-twice BP for IP_2 on n variables. Define the noncommutative Fourier transform $F(P)$ of P 's transition matrix as the matrix-valued function $F(P)(g) = \sum_{\mathbf{x}} \{ \mathbf{x} \} P(\mathbf{x}) * g(\mathbf{x})$, where g is a noncommutative character. Then, the operator norm $\|F(P)\|_{\text{op}} = \Theta(\log n)$ if P is a trivial BP (constant function), but $\|F(P)\|_{\text{op}} = \Omega(n)$ for any read-twice BP computing IP_2.

Rationale (proposer's reasoning):

Noncommutative harmonic analysis captures the algebraic structure of branching programs via matrix-valued characters, revealing inherent dimensionality gaps between read-twice and read-once models. The operator norm of the Fourier transform quantifies the 'spread' of transition probabilities across noncommutative representations, exposing structural constraints unique to read-twice BPs.

Taxonomy category: BARRINGTON_ALG (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a56c41cf0323d9d9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=241.2s

5.1 Generated Python source

```
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda j: abs(A[j][i]))
            A[i], A[max_row] = A[max_row], A[i]
            factor = Fraction(A[i][i])
            for j in range(n):
                A[i][j] /= factor
            for k in range(m):
                if k != i:
                    factor = Fraction(A[k][i])
                    for j in range(n):
                        A[k][j] -= factor * A[i][j]
        return A

    def matrix_multiply(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[Fraction(0) for _ in range(p)] for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def operator_norm(A):
        m, n = len(A), len(A[0])
        max_row_sum = 0
        for i in range(m):
            row_sum = sum(abs(x) for x in A[i])
            if row_sum > max_row_sum:
                max_row_sum = row_sum
```

```

    return max_row_sum

def noncommutative_fourier_transform(P, n):
    m = len(P)
    F = [[Fraction(0) for _ in range(m)] for _ in range(m)]
    for x in range(m):
        g_x = [Fraction(x & (1 << i)) for i in range(n)]
        F[x] = matrix_multiply(P, g_x)
    return F

def read_twice_bp(n):
    P = [[0] * n for _ in range(2 ** n)]
    for x in range(2 ** n):
        if x & 1 == 0:
            P[x][x ^ (1 << (n - 1))] = Fraction(1)
        else:
            P[x][x ^ (1 << (n - 1))] = Fraction(-1)
    return P

def is_trivial_bp(P):
    m, n = len(P), len(P[0])
    for x in range(m):
        if sum(abs(p) for p in P[x]) != 1:
            return False
    return True

n = 40
P = read_twice_bp(n)
F = noncommutative_fourier_transform(P, n)
norm = operator_norm(F)

metric_name = "operator_norm"
metric_value = norm
instances_tested = 1
conjecture_holds = is_trivial_bp(P) and norm == Fraction(2 * n - 2) or not is_trivial_bp(P) and
counterexample = "" if conjecture_holds else "Nontrivial BP with small norm"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_norm = sum(r["metric_value"] for r in results) / len(results)

```

```

support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_norm} std=0.0 support_fraction=1.0")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_norm} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Increase timeout duration and re-run test with extended computation limits

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	136685
2	propose	ollama_remote	qwen3:8b	0	0	106667
3	propose	ollama_remote	qwen3:8b	0	0	121559
4	preregistration	ollama_remote	qwen3:8b	0	0	31803
5	novelty	ollama_remote	qwen3:8b	0	0	27214
6	novelty	ollama_remote	qwen3:8b	0	0	22239
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17940
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20821
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	28163
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	49757
11	judge	ollama_remote	qwen3:8b	0	0	112779

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 675627 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in [research/audit/713f57619482.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/713f57619482.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/713f57619482.tar.gz` (if generated)